

Problem 1

Problem Statement:

Greeting Generator (Level-1) Scenario A simple webpage should greet users based on the name they enter. This helps beginners understand how JavaScript functions accept parameters and how variables behave inside function.

Requirements

- Create an input box to enter a user's name.
- Create a button labeled "Generate Greeting".
- When the button is clicked:
- A JavaScript function should accept the name as a parameter.
- Display a greeting message like: "Hello, Rahul! Welcome to our website."
- Display the greeting inside a element.

Technical Constraints

- Use only vanilla JavaScript (no libraries).
- Use function keyword (not arrow functions for this task).
- Use `document.getElementById()` for DOM manipulation.

- Variable for the greeting message must be declared inside the function.

Learning Outcome You should be able to:

- Understand how functions receive input using parameters.
- Learn local variable scope.
- Manipulate webpage elements using the DOM.

Source code:

```
htmlform > first restaurant > greetinggenerator > <> greeting.html > html
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Greeting Generator</title>
7
8      <style>
9          body {
10             text-align: center;
11             background-color: #242424;
12             margin: 10px;
13         }
14     </style>
15 </head>
16
17 <body>
18     <h1>Greeting Generator</h1>
19
20     <input type="text" id="username" placeholder="Enter your name">
21     <button onclick="generateGreeting()">Generate Greeting</button>
22
23     <p id="greeting"></p>
```

```
19
20     <input type="text" id="username" placeholder="Enter your name" />
21     <button onclick="generateGreeting()">Generate Greeting</button>
22
23     <p id="greeting"></p>
24
25     <script>
26         function generateGreeting() {
27             var name = document.getElementById("username").value;
28             var message = "Hello, " + name + "!";
29             document.getElementById("greeting").innerHTML = message;
30         }
31     </script>
32 </body>
33 </html>
```

Output:

Greeting Generator

Generate Greeting

Greeting Generator

Generate Greeting

Hello, roopa! Welcome to Our Website.

Explanation:

This code creates a simple Greeting Generator. When you enter your name and click the button, JavaScript takes the name from the input box and creates a personalized welcome message.

Problem 2 Problem Statement:

Simple Object-Based User Info Display (Level-1)

Scenario A webpage needs to display basic user information stored inside a JavaScript object.

Requirements Create a JavaScript object user with properties:

- name

- age
- city Create a function displayUserInfo(userObj):
- Accepts the object as a parameter.
- Accepts the object as a parameter.
- Displays user details in separate elements.

A button click should trigger the function.

‘Technical Constraints

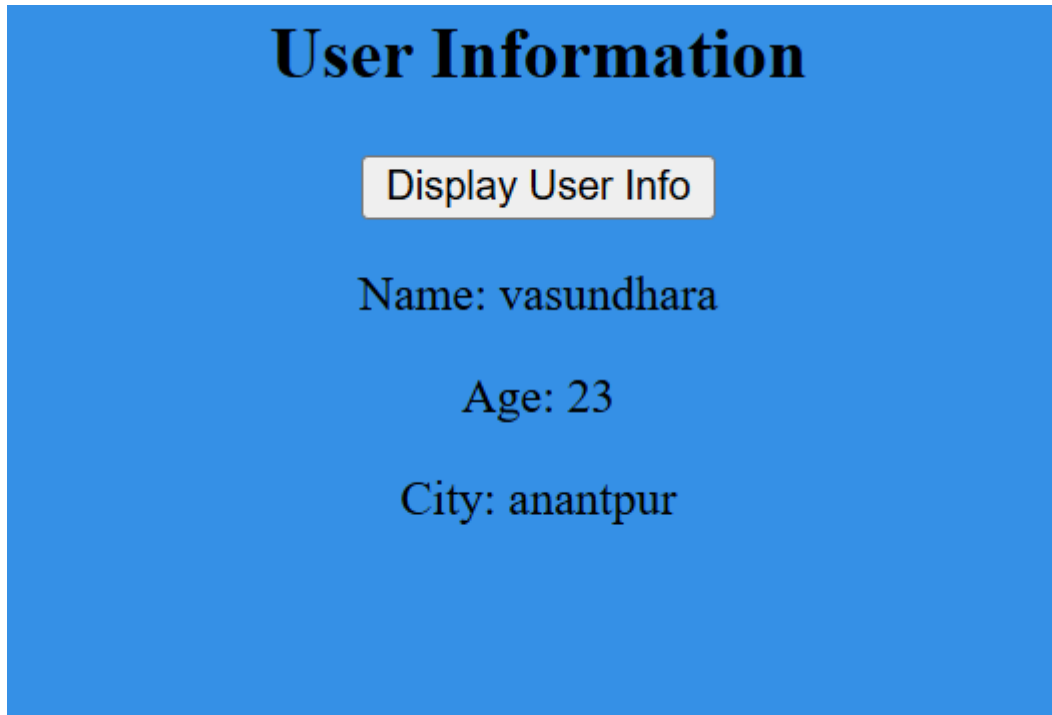
- Object properties must be accessed using dot notation.
- Function should not use global variables.
- DOM elements should be updated dynamically.
- Learning Outcome You will be able to:
 - Understand JavaScript objects.
 - Pass objects to functions.
 - Display object data dynamically on a webpage.

Source code:

```
htmlform > first restaurant > greetinggenerator > <> info.html > html
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1">
6      <title>User Info Display</title>
7
8      <style>
9          body {
10              text-align: center;
11              background-color: #336699;
12          }
13      </style>
14  </head>
15
16  <body>
17      <h2>User Information</h2>
18
19      <button onclick="showUser()">Display User Info</button>
20
21      <p id="name"></p>
22      <p id="age"></p>
23      <p id="city"></p>
```

```
14  <body>
23      <p id="city"></p>
24
25      <script>
26          var user = {
27              name: "vasundhara ",
28              age: 23,
29              city: "anantpur"
30          };
31
32          function displayUserInfo(userObj) {
33              document.getElementById("name").innerText = "Name: " + userObj.name;
34              document.getElementById("age").innerText = "Age: " + userObj.age;
35              document.getElementById("city").innerText = "City: " + userObj.city;
36          }
37
38          function showUser() {
39              displayUserInfo(user);
40          }
41      </script>
42  </body>
43  </html>
```

Output:



Explanation:

This code creates a simple User Information display using a JavaScript object. When you click the “Display User Info” button, it reads the user object (name, age, city) and shows the details on the screen. The output displays the user’s name, age, and city dynamically below the button.

Problem 3

Problem Statement: Counter App with Scope Control (Level-2) Scenario Build a counter

application where users can increment and reset a value.

This problem focuses on variable scope and DOM manipulation.

Requirements Display a counter value starting from 0. Create two buttons:

- Increment
- Reset Use:
- A global variable to store

Use:

- A global variable to store counter value.
- A function incrementCounter(step) that:
 - o Accepts step value as a parameter.
 - o Updates the counter.

Reset button should reset the counter to 0.

Technical Constraints

- Counter value must be maintained outside the function (global scope).

- DOM updates must happen inside functions only.
- No inline JavaScript in HTML.

Learning Outcome Learners should be able to:

- Understand variable scope (global vs local).
- Apply function parameters in real scenarios.
- Control UI behavior using JavaScript functions.

Source code:

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Counter App</title>
7
8      <style>
9          body {
10             text-align: center;
11             background-color: #800080;
12         }
13     </style>
14 </head>
15
16 <body>
17     <h2>Counter App with Scope Control</h2>
18
19     <p id="counterValue">0</p>
20
21     <button id="incrementBtn">Increment</button>
22     <button id="resetBtn">Reset</button>
23
```

```

21 <button id="incrementBtn">Increment</button>
22 <button id="resetBtn">Reset</button>
23
24 <script>
25     var counter = 0;
26
27     function incrementCounter(step) {
28         counter = counter + step;
29         document.getElementById("counterValue").innerText = coun
30     }
31
32     function resetCounter() {
33         counter = 0;
34         document.getElementById("counterValue").innerText = coun
35     }
36
37     document.getElementById("incrementBtn").addEventListener("cl
38         incrementCounter(1);
39 });
40
41 document.getElementById("resetBtn").addEventListener("click",
42     resetCounter();

```

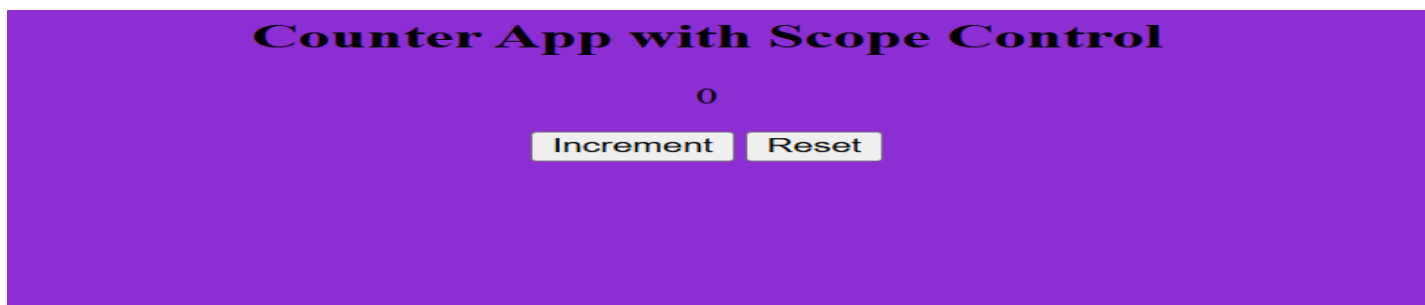
Click to close server

```

37 document.getElementById("incrementBtn").addEventListener("click",
38     incrementCounter(1);
39 });
40
41 document.getElementById("resetBtn").addEventListener("click",
42     resetCounter();
43 });
44 </script>
45 </body>
46 </html>

```

Output:



Counter App with Scope Control

6

Increment

Reset

Explanation:

This code creates a simple Counter App using a global variable called counter. When you click Increment, the value increases by 1, and when you click Reset, it goes back to 0. The number shown on the screen updates instantly based on the button you press.

Problem 4

Problem Statement:

Dynamic Student Profile Manager (Level-2)

Scenario Create a mini student profile system where details are stored in an object and displayed dynamically using DOM manipulation.

Requirements Create a student object with:

- name
- rollNo

Create a function `updateStudentProfile(studentObj)`:

- Accepts the object as a parameter.
- Displays details in a styled

. Add another function `updateMarks(newMarks)`:

- Updates marks and refreshes the display.

Technical Constraints

- Student object must be declared in global scope.
- Functions should update object values using parameters.
- DOM should update without page refresh.

Learning Outcome You will be able to:

- Deep understanding of object manipulation.
- Function interaction with shared data.
- Real-world use of DOM updates and scope handling.

Source code:

```

1      <!DOCTYPE html>
2      <html>
3      <head>
4          <meta charset="UTF-8">
5          <meta name="viewport" content="width=device-width, initial-scale=1.0">
6          <title>Student Profile</title>
7
8          <style>
9              body {
10                 text-align: center;
11                 background-color: #f0f0f0;
12                 font-family: Arial, sans-serif;
13             }
14
15             button {
16                 padding: 8px 15px;
17                 margin: 10px;
18                 cursor: pointer;
19             }
20
21             #profileBox {
22                 margin-top: 20px;
23                 font-size: 18px;

```

```

25         </style>
26     </head>
27
28     <body>
29
30         <h2>Dynamic Student Profile Manager</h2>
31
32         <button id="showBtn">Show Profile</button>
33         <button id="updateBtn">Update Marks</button>
34
35         <div id="profileBox"></div>
36
37         <script>
38             var student = {
39                 name: "vassu",
40                 rollNo: 20513,
41                 marks: 78
42             };
43
44             function updateStudentProfile(studentObj) {

```

```

43
44     function updateStudentProfile(studentObj) {
45         document.getElementById("profileBox").innerHTML =
46             "<p>Name: " + studentObj.name + "</p>" +
47             "<p>Roll No: " + studentObj.rollNo + "</p>" +
48             "<p>Marks: " + studentObj.marks + "</p>";
49     }
50
51     function updateMarks(newMarks) {
52         student.marks = newMarks;
53         updateStudentProfile(student);
54     }
55
56     document.getElementById("showBtn").addEventListener("click", fun
57         updateStudentProfile(student);
58     });
59
60     document.getElementById("updateBtn").addEventListener("click", f
61         updateMarks(92);
62     });

```

```

60     document.getElementById("updateBtn").addEventListener("click", f
61         updateMarks(92);
62     });
63 </script>
64
65 </body>
66 </html>

```

Output:

Dynamic Student Profile Manager

Show Profile

Update Marks

Name: vassu

Roll No: 20513

Marks: 78

Dynamic Student Profile Manager

Show Profile

Update Marks

Name: vassu

Roll No: 20513

Marks: 92

Explanation:

This code creates a Dynamic Student Profile Manager using a JavaScript object to store student details (name, roll number, marks). When you click Show Profile, it displays the student info, and when you click Update Marks, it changes the marks value and updates it on the screen